AIR VISUAL INTERNAL DOCUMENT

Vision Models, Workflow, and Decisions

This document summarizes the vision understanding models, image-generation model usage, end-to-end workflow, and design decisions reflected in the current Air Visual repository.

Prepared from the codebase state on 2026-03-18. Repo evidence came from `README.md`, `server/server.mjs`, `content.js`, `sidepanel.js`, and `scripts/generate_cws_assets.py`.

Evidence Note

> The repo does not currently preserve a separate experiment log, benchmark sheet, or decision journal. As a result, this write-up documents the models and decisions that are clearly reflected in current code, README defaults, and supported execution paths. It should be treated as an implementation-grounded summary, not a retrospective lab notebook.

Executive Summary

- Air Visual's core product path is vision understanding, not generative photo editing.
- The hosted analysis default is `gemini-2.5-flash-lite`.
- The local analysis default and main Ollama path is `qwen2.5vl:3b`.
- `moondream` is retained as a lighter local fallback for constrained machines.
- The backend also supports multi-model Ollama consensus when multiple compatible local vision models are installed.
- Generated image output is used only for Chrome Web Store assets via `gemini-2.5-flash-image`, not for Airbnb listing photo edits.
- The product deliberately stays inside a truthful Tier 1 boundary: cover choice, light, crop, composition, amenity visibility, and sequence.

Models Covered

Area	Model	Repo Status	What It Is Used For	Decision Summary
Vision understanding	`gemini-2.5-flash-lite`	Hosted default in README and backend	Analyze listing photos and return structured recommendation JSON through the Gemini Developer API.	Kept as the low-cost hosted path for fast setup, predictable JSON output, and no local model install requirement.
Vision understanding	`qwen2.5vl:3b`	Default Ollama model and fallback path	Primary local image-analysis model for hosts who want an on-device path.	Kept as the practical local default because it balances quality and local usability better than heavier models for this MVP.
Vision understanding	`moondream`	Supported lighter fallback	Constrained-machine fallback when a smaller local model is needed.	Retained for speed and lower hardware pressure, but the code narrows it to a smaller first pass because quality is treated as lower-confidence.
Vision understanding	`qwen2.5vl:7b` and other Ollama vision models matched by `isVisionModel()`	Supported, not default	Optional multi-model consensus runs when multiple installed local vision models are configured.	Supported as an extensibility path, but not promoted as the default because the MVP is optimizing for operational simplicity.
Image generation	`gemini-2.5-flash-image`	Default in the asset-generation script	Generate Chrome Web Store icon, promo art, and screenshot-style marketing assets from a reference image.	Used only for store and marketing assets. Explicitly not part of the listing-photo recommendation workflow.

Important distinction: the repo does not implement generative editing of Airbnb listing photos. Preview changes inside the product are deterministic CSS-style filter and crop simulations.

Understanding Workflow

1. The Chrome extension detects an Airbnb listing or host page and extracts page context from the active tab.
2. The extracted payload includes title, listing URL, visible photo candidates, nightly rate when available, occupancy hints when available, and an amenities preview.
3. The side panel checks backend readiness through `/api/status` before starting analysis.
4. The backend chooses the active path: Gemini when `AI_PROVIDER=gemini` or when `AI_PROVIDER=auto` and `GEMINI_API_KEY` is present; otherwise Ollama.
5. The backend fetches listing image bytes from the source URLs and converts them to base64 for model input.
6. Gemini receives the prompt plus inline image data and returns schema-constrained JSON.
7. Ollama receives the prompt, attached base64 images, and the same schema target. If multiple configured local vision models are installed, the backend can run them in parallel and merge results by vote threshold.
8. The backend normalizes results, prunes duplicate recommendations, sanitizes edit plans, estimates revenue impact, and ranks recommendations by priority.
9. The side panel renders recommendations, deterministic preview variants, or sequence-move previews.
0. User decisions are stored locally in `chrome.storage.local` as accept or decline markers tied to listing URL and recommendation ID.

Generation Workflow

1. A separate script, `scripts/generate_cws_assets.py`, handles image generation for Chrome Web Store assets.
2. The script uses a provided reference logo or brand image as conditioning input.
3. It calls Gemini image generation with a branded prompt and asset-specific copy rules.
4. Generated master images are written to `tmp/generated-cws-masters/`.
5. The script then resizes those masters into the required icon, promo, and screenshot deliverables inside `icons/` and `store-assets/`.
6. The README already notes that generated screenshot-style assets are less trustworthy than real captures for final store submission.

Decisions Made

1. Keep the product inside a truthful Tier 1 boundary

The system only recommends plausible changes such as cover selection, lighting, crop, composition, amenity visibility, and sequence. It explicitly avoids fabricated amenities, room changes, or fantasy edits.

2. Use model output for analysis, not for end-user image rewriting

The recommendation engine can call hosted or local vision models, but user-facing previews are deterministic transforms. This reduces product risk, keeps changes explainable, and stays aligned with the truthful-edit boundary.

3. Prefer a simple hosted default and a practical local default

`gemini-2.5-flash-lite` is the hosted default because it is lightweight and easy to stand up. `qwen2.5vl:3b` is the local default because it is the main operational compromise between cost, availability, and acceptable vision performance for an MVP.

4. Keep a lighter fallback for weaker local hardware

`moondream` is still supported, but the code intentionally constrains it to a smaller first pass. That is a clear sign the repo treats it as a speed and hardware fallback, not as the preferred quality path.

5. Add consensus only where it is operationally justified

Multi-model consensus exists for Ollama, not for Gemini. The backend groups overlapping recommendations, requires a configurable minimum vote count, and keeps only consensus items. This reflects a decision to spend extra local compute only when several local models are available.

6. Limit photo count and timeout windows by model path

Photo count and timeout logic are explicitly model-aware. This keeps the extension responsive and recognizes that small local models, larger local models, and hosted Gemini have different latency envelopes.

> **7. Gate hero-image replacement more strictly than simple edits**
>
> Cover-photo replacement is only allowed under tighter conditions. When confidence or context is not strong enough, the system downgrades the idea into crop, brightness, or composition guidance instead of forcing a hero swap.

> **8. Keep image generation isolated to brand and store materials**
>
> The repo uses Gemini image generation for icons and marketing graphics, but it does not use generated imagery to alter listing photos. That separation is deliberate and consistent with the product's trust model.

Current Recommendation

- Keep `gemini-2.5-flash-lite` and `qwen2.5vl:3b` as the main comparison pair for understanding tasks.
- Treat `moondream` as a machine-constraint fallback, not a primary quality baseline.
- Keep `gemini-2.5-flash-image` scoped to marketing asset generation only.
- If future testing becomes more formal, add a benchmark set of real listing pages, failure cases, latency numbers, and a decision log to the repo.

Open Gaps

- No benchmark dataset is checked into the repo.
- No quantitative quality comparison between Gemini and Ollama models is preserved yet.
- No historical experiment notes explain when or why a specific model was added or removed.
- No final submission-grade store screenshots are preserved alongside the generated marketing set.